

# PFS004 - SIGEA

Developement Team

2025-11-15

## Indice

|                                                                               |          |
|-------------------------------------------------------------------------------|----------|
| <b>Estado</b>                                                                 | <b>1</b> |
| <b>I. Modificacion a la Estructura actual del proyecto</b>                    | <b>1</b> |
| 1.1. Estructura del Proyecto Modificada . . . . .                             | 1        |
| 1.2. Explicacion de los Cambios . . . . .                                     | 3        |
| 1.2.1. Adicion de requestDT0 y responseDT0 en presentation/ . . . . .         | 3        |
| 1.2.2. Adicion de serviceDT0 en services/ y mapper en presentation/ . . . . . | 4        |
| 1.2.3. Adicion de security/ . . . . .                                         | 4        |

## Estado

[Aceptado]

## I. Modificacion a la Estructura actual del proyecto

Conforme fue avanzando el desarrollo surgieron nuevos problemas e inconvenientes que nos llevaron a modificar ligeramente la estructura planteada en PFS003.

La nueva estructura es la siguiente:

### 1.1. Estructura del Proyecto Modificada

```

app/
  src/
    main/
      java/
        com/zentry/sigea/
          config/          # Configuración de Spring, Beans, Seguridad, CORS, etc
          utils/           # Utilidades genéricas (fechas, strings, etc.)
          security/        # Componentes y Servicios que controlan la autenticación
          presentation/    # Capa de presentación (UI + Controladores)
            api/           # @RestController comunicacion con frontend
            models/        # DTOs y ViewModels que la vista necesita
              requestDTO/  # DTOs para mapear y validar solicitudes desde el frontend
              responseDTO/ # DTOs para enviar datos de regreso al frontend
              mapper/      # mapean request -> domain/serviceDTO o domain/servicio
          services/        # Capa de aplicación (Casos de uso)
            usecases/      # Casos de uso concretos
            interfaces/    # Interfaces para los servicios del dominio
            serviceDTO/    # DTOs específicos para la capa de "services"
          core/            # Capa de dominio
            entities/      # Entidades de dominio
            value-objects/ # Objetos de valor (si aplica)
            repositories/  # Interfaces de los repositorios
            services/      # Servicios de dominio puros
          infrastructure/  # Capa de infraestructura
            repositories/  # Implementaciones de los repositorios
            database/      # Configuración y acceso a base de datos
              entities/    # Entidades JPA (@Entity)
              mappers/     # Mappers entre dominio y infraestructura
            external-apis/ # Clientes de APIs externas
            config/        # Configuración de infraestructura (env, logger, etc)
        resources/
          application.properties # Configuración principal (por defecto)
          application.yml        # O alternativa en formato YAML
          i18n/                  # Archivos de internacionalización (mensajes por idioma)
            messages_en.properties
            messages_es.properties
          db/
            migration/          # Scripts SQL para migraciones (ej: Flyway, Liquibase)
              V1__init.sql

```

```

        seed/                                # Datos de prueba para poblar la DB
        logback-spring.xml                  # Configuración del sistema de logs (Logback)
        banner.txt                          # Banner que se muestra al arrancar la app
        META-INF/                          # Archivos especiales como manifest, spring.factories
test/
    java/
        com/miempresa/myapplication/
            presentation/
            services/
            core/
            infrastructure/
            ...
docs/

```

## 1.2. Explicacion de los Cambios

### 1.2.1. Adicion de requestDTO y responseDTO en presentation/

```

app/
    src/
        main/
            java/
                com/zentry/sigea/
                    presentation/              # Capa de presentación (UI + Controladores)
                        models/              # DTOs y ViewModels que la vista necesita
                            requestDTO/      # DTOs para mapear y validar solicitudes desde el
                            responseDTO/     # DTOs para enviar datos de regreso al frontend
                            ...

```

Este cambio se implemento para poder separar de forma clara las clases que mapen solicitudes y respuestas desde y hacia el frontend. Las clases en **requestDTO** nos permiten recibir solicitudes mas complejas y variadas, ademas, con la implementacion de la dependencia **spring-boot-starter-validation**, podemos validar cada campo y enviar mensajes amigables al usuario.

Las clases en **responseDTO** nos permiten, de igual manera que con **requestDTO**, enviar respuestas mas elaboradas que un sencillo mensaje de operacion exitosa o fallida.

### 1.2.2. Adición de serviceDTO en services/ y mapper en presentation/

```
app/  
  src/  
    main/  
      java/  
        com/zentry/sigea/  
          presentation/      # Capa de presentación (UI + Controladores)  
            models/          # DTOs y ViewModels que la vista necesita  
              mapper/        # mapean request -> domain/serviceDTO o domain/ser  
                ...  
  
          services/          # Capa de aplicación (Casos de uso)  
            interfaces/      # Interfaces para los servicios del dominio  
              ...
```

Este cambio en principio puede ser visto como innecesario, debido a que **services/** podría hacer uso de los **requestDTO** o **responseDTO** directamente de **presentation/** para sus operaciones, pero esto rompe el concepto fundamental de la arquitectura en capas implementada, donde una capa inferior no puede depender de una capa superior (flujo unidireccional).

Así que, para reducir el acoplamiento y dependencia mutua entre **services/** y **presentation/**, se añade **serviceDTO** (**services/**) el cual contiene DTOs específicos para las operaciones de **service/** libres de cualquier referencia a **presentation/**, además, se añade **mapper** (**presentation/**) para poder mapear los **requestDTO** a **domain/serviceDTO**, o **domain/service** a **responseDTO**.

No es necesario declarar un **serviceDTO** por cada **requestDTO**, ya que, dependiendo de la información que contenga un **requestDTO**, este podrá ser mapeado a un **DomainEntity**. Pero si la información que contiene un **requestDTO** es demasiado compleja como para poder ser mapeada a uno o varios **DomainEntity** entonces se puede usar un **serviceDTO**.

### 1.2.3. Adición de security/

```
app/  
  src/  
    main/  
      java/  
        com/zentry/sigea/  
          security/          # Componentes y Servicios que controlan la autenticación
```

`security/` fue específicamente pensada para contener componentes y servicios que controlan las configuraciones o aspectos críticos de JWT, para asegurar una correcta autenticación de usuarios mediante tokens temporales, con el fin de restringir el acceso según roles (ADMINISTRADOR, ORGANIZADOR, PARTICIPANTE) a determinados endpoints del backend.